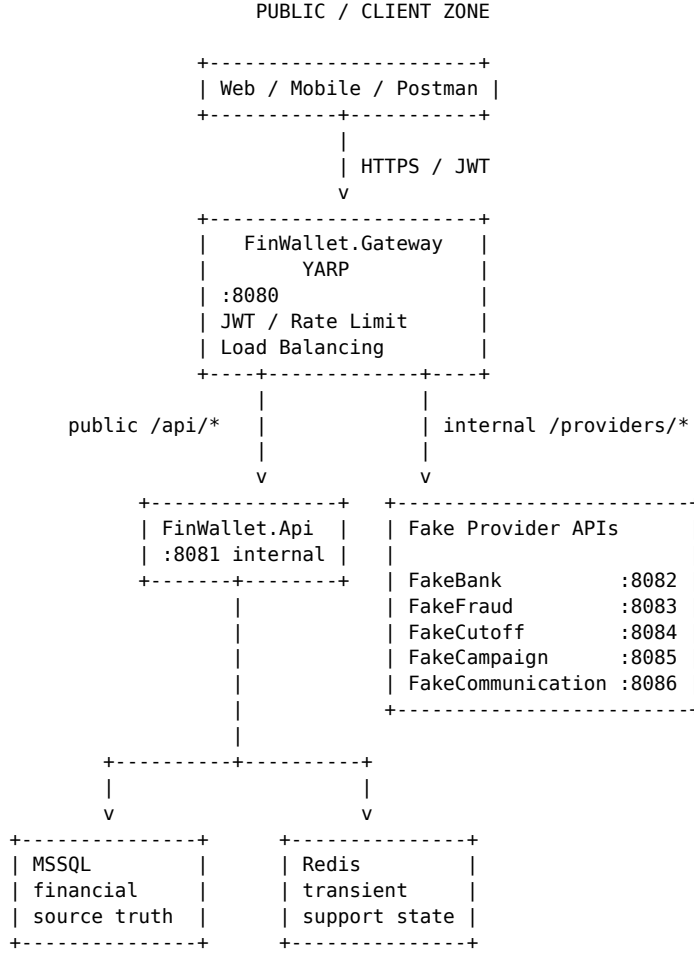


FinWallet Sistem Topolojisi

1. Amaç

Bu belge FinWallet v1'in çalışma zamanındaki servis, güvenlik, veri, finansal işlem ve Docker topolojisini tek yerde gösterir. Amaç projeyi ilk kez gören bir geliştiricinin hangi bileşenin nerede çalıştığını, trafiğin hangi güven sınırlarından geçtiğini, hangi verinin nerede tutulduğunu ve hangi parçaların birlikte ölçeklendiğini hızlıca anlayabilmesidir.

2. Yüksek seviye runtime topolojisi



Normal client yalnız Gateway'i çağırır. FinWallet.Api ve Fake provider servisleri public giriş noktası değildir.

3. Uygulama katman topolojisi



Shared:

FinWallet.Shared.Contracts -> ServiceResult ve ortak transport sözleşmeleri
FinWallet.Shared.Web -> Swagger, rate limit, CORS, Kestrel limitleri, security header'ları, internal service-key kontrolü

Dependency yönü:

Api -> Application -> Domain
Api -> Infrastructure -> Application / Domain
Gateway -> Shared.Web / Shared.Contracts
Fake APIs -> Shared.Web / Shared.Contracts

Domain SQL, HTTP, Redis veya provider DTO bilmez. Application concrete Infrastructure implementasyonuna bağımlı değildir.

4. Gateway ve trust boundary topolojisi

```
[Client]
|
| JWT
v
[Gateway]
|
| JWT + DownstreamServiceKey
v
[FinWallet.Api]

[FinWallet.Api]
|
| InternalServiceKey
v
[Gateway /providers/*]
|
| DownstreamServiceKey
v
[Fake Provider]
```

Bu model dört ayrı kontrol noktası oluşturur:

- 1 Client -> Gateway: public edge authentication ve rate-limit.
- 2 Gateway -> FinWallet.Api: Gateway ayrıca downstream service credential ekler; API JWT ve ownership kontrolünü tekrar yapar.
- 3 FinWallet.Api -> Gateway provider route: InternalServiceKey ile yalnız trusted internal caller kabul edilir.
- 4 Gateway -> Fake provider: ayrı DownstreamServiceKey destination service tarafından doğrulanır.

Sonuç: Gateway bypass edilip doğrudan backend portuna erişilse bile business endpoint otomatik olarak güvenilir hale gelmez.

5. YARP route ve load-balancing topolojisi

```
Client /api/*
|
v
route: finwallet-api
|
v
cluster: finwallet-api-cluster
|
+--> FinWallet.Api replica 1
+--> FinWallet.Api replica 2 (production eklenebilir)
+--> FinWallet.Api replica N

FinWallet.Api /providers/bank/*
|
v
route: fake-bank
|
v
cluster: fake-bank-cluster
```

```
|
+--> FakeBank replica 1
+--> FakeBank replica N
```

YARP cluster'larında load-balancing policy PowerOfTwoChoices olarak yapılandırılabilir. Destination adresleri, health-check, max connection ve timeout değerleri appsettings üzerinden yönetilir. Development ortamında tek destination yeterlidir; production ortamında aynı cluster'a replica eklenebilir.

6. Public API trafik topolojisi

```
Client
|
| POST /api/v1/auth/register
| POST /api/v1/auth/login
| POST /api/v1/wallets
| POST /api/v1/bank-accounts
| POST /api/v1/bank-movements/deposits
| POST /api/v1/transfers
| POST /api/v1/purchases
v
Gateway
|
v
FinWallet.Api Controllers
|
v
Application Handler
|
+--> Domain rules
+--> MSSQL / Redis
+--> gerektiğinde Gateway /providers/*
```

Anonymous route'lar register, registration verify, login ve refresh ile sınırlıdır. Diğer müşteri endpoint'lerinde Gateway JWT ister.

7. Provider entegrasyon topolojisi

```
FinWallet.Api
|
+--> /providers/bank/* -----> FakeBank.Api
|
+--> /providers/fraud/* -----> FakeFraud.Api
|
+--> /providers/cutoff/* -----> FakeCutoff.Api
|
+--> /providers/campaign/* -----> FakeCampaign.Api
|
+--> /providers/communication/* -> FakeCommunication.Api
```

Provider sorumlulukları:

Servis	Sorumluluk
FakeBank.Api	Dış banka hesabı, provider transaction, statement, timeout/failure simülasyonu
FakeFraud.Api	External Allow/Review/Deny, score ve reason code
FakeCutoff.Api	Business day, cutoff, processing ve settlement tarihi
FakeCampaign.Api	Merchant campaign eligibility, discount ve sponsor tipi
FakeCommunication.Api	SMS/e-posta kabul, delay, timeout ve failure simülasyonu

Provider DTO'ları Application/Domain'e sızmaz; Infrastructure adapter/anti-corruption layer içinde map edilir.

8. Veri sahipliği topolojisi

```
+-----+
| MSSQL - AUTHORITATIVE |
+-----+
```

Customer	
Credential / Session	
Wallet / BankAccount	
FinancialTransaction	
LedgerJournal / LedgerEntry	
Idempotency	
FraudEvent / Review	
Outbox / Inbox	
Reconciliation	
Audit	
+-----+	
+-----+	
Redis - TRANSIENT SUPPORT	
+-----+	
OTP TTL	
temporary counters	
fraud velocity	
hot/session cache support	
temporary coordination	
+-----+	

Finansal doğruluk için Redis tek otorite değildir. Wallet balance, ledger, completed transaction ve durable idempotency MSSQL'de tutulur.

9. Bankadan Wallet'a para giriş topolojisi

```

Customer Bank Account (FakeBank)
|
| provider withdrawal
v
FinWallet Bank Settlement Asset
|
| same DB transaction
+--> Customer Wallet Liability +1000
+--> FinancialTransaction = Completed
+--> LedgerJournal
+--> LedgerEntries (Debit = Credit)
+--> Idempotency final result
+--> Outbox notification

```

Önemli ayrım: BankAccount ile Wallet aynı şey değildir. BankAccount dış banka hesabını; Wallet FinWallet içindeki müşteri bakiyesini temsil eder.

10. Wallet-to-Wallet transfer topolojisi

```

Client
|
v
Gateway JWT
|
v
FinWallet.Api
|
+--> completed idempotency replay check
+--> server-side risk signals
+--> Internal Fraud
+--> Gateway -> FakeFraud
+--> Allow / Review / Deny
|
+--> ALLOW ise tek MSSQL transaction:
    source Wallet debit
    destination Wallet credit
    FinancialTransaction
    LedgerJournal
    LedgerEntries
    durable Idempotency result

```

External fraud HTTP çağrısı açık SQL transaction içinde yapılmaz.

11. Purchase topolojisi

```

Client

```

```

|
v
Gateway -> FinWallet.Api
|
+--> internal fraud
+--> external FakeFraud
+--> FakeCampaign
|
v
Atomic MSSQL posting
|
+--> customer wallet liability debit
+--> merchant payable credit
+--> campaign expense debit (platform sponsor ise)
+--> FinancialTransaction
+--> balanced Ledger
+--> Outbox

```

Campaign provider yalnız indirim/uygunluk bilgisini hesaplar; finansal muhasebe FinWallet tarafından yapılır.

12. Background worker topolojisi

```

FinWallet.Api process
|
+--> BankMoneyMovementBackgroundService
|   |
|   +--> due Scheduled/Pending transactions
|   +--> Gateway -> FakeBank status / processing
|   +--> durable finalization / compensation
|
+--> OutboxDispatchBackgroundService
|   |
|   +--> claim Outbox row
|   +--> SQL transaction closes
|   +--> Gateway -> FakeCommunication
|   +--> success/failure/backoff state

```

Notification başarısızlığı tamamlanmış para hareketini rollback etmez. Outbox yeniden dener.

13. Inbox callback topolojisi

```

FakeBank / test provider
|
| X-Internal-Service-Key
v
Gateway InternalService route
|
v
/api/v1/internal/bank/callbacks
|
v
Inbox dedupe (Source + MessageId)
|
v
Bank movement finalization

```

Aynı callback tekrar gelirse Inbox ve terminal financial state duplicate money movement oluşmasını engeller.

14. Reconciliation topolojisi

```

Internal Reconciliation API
|
+--> Wallet <-> Ledger-derived balance
|
+--> Bank transaction <-> Settlement ledger
|
+--> FinWallet <-> FakeBank statement
|
v
ReconciliationRun + ReconciliationIssue

```

Reconciliation fark bulunduğunda bakiyeyi otomatik düzeltmez; issue üretir.

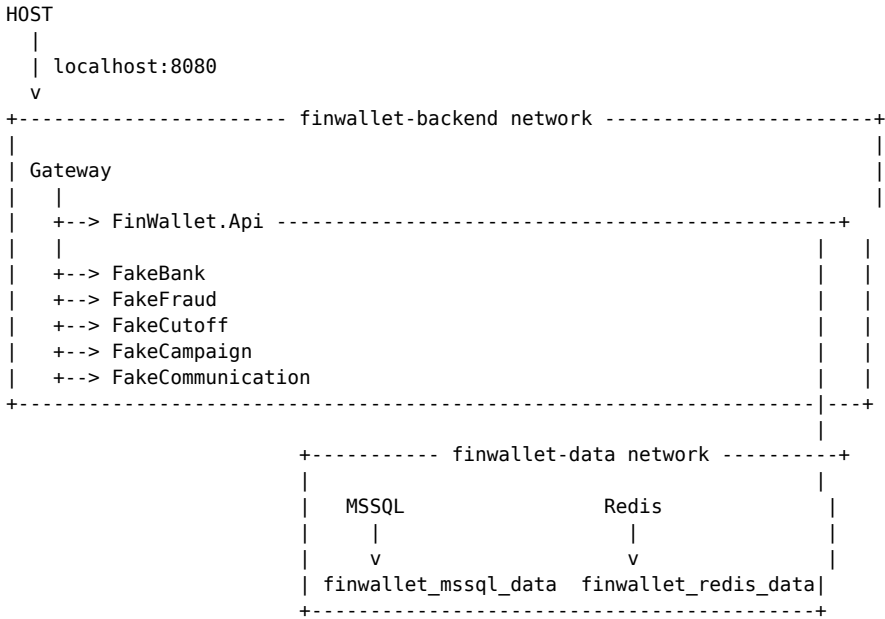
15. Swagger topolojisi

Swagger ortak FinWallet.Shared.Web üzerinden aşağıdaki HTTP projelerinde vardır:

```
FinWallet.Gateway
FinWallet.Api
FakeBank.Api
FakeFraud.Api
FakeCutoff.Api
FakeCampaign.Api
FakeCommunication.Api
```

Development'da açılabilir; production'da varsayılan kapalıdır. Swagger açık olması endpoint authorization'ını bypass etmez.

16. Docker runtime topolojisi



Normal Compose kullanımında host'a yalnız Gateway 8080 publish edilir. Backend API'ler, MSSQL ve Redis Docker network içinde kalır. Debug overlay gerektiğinde backend portlarını yalnız 127.0.0.1 üzerinden açar.

17. Docker persistent volume topolojisi

```
mssql container
+--> finwallet_mssql_data : database data/log
+--> finwallet_mssql_backup : backup files

redis container
+--> finwallet_redis_data : AOF/RDB persistence

application containers
+--> named persistent application volume YOK
      stdout/stderr -> Docker log driver
```

docker compose down volume'ları korur. docker compose down -v MSSQL ve Redis local persistence'ını siler.

18. Ölçekleme sınırları

Gateway, FinWallet.Api ve stateless provider HTTP servisleri replica olarak çoğaltılabilir. Finansal correctness replica sayısına bağlı değildir; MSSQL constraint/transaction/idempotency kuralları multi-instance correctness sağlamalıdır.

Redis distributed financial lock olarak kullanılmaz. Bir instance kaybolduğunda committed financial state MSSQL'de kalır.

19. Finansal consistency özeti

```
External HTTP
|
| SQL transaction açık DEĞİL
v
Provider result
|
v
Short MSSQL transaction
|
+--> Wallet / BlockedBalance
+--> FinancialTransaction
+--> Ledger
+--> Idempotency
+--> Outbox
v
COMMIT
```

Bu sınır distributed transaction ihtiyacını azaltır ve para hareketinin yarım commit edilmesini engeller.

20. Bir isteği izlemek için referanslar

Uçtan uca troubleshooting sırasında aşağıdaki referanslar birlikte kullanılır:

```
X-Correlation-Id
CustomerId
SessionId
TransactionId
BankReference / ExternalTransactionId
FraudReference
CampaignReference
CutoffReference
Outbox / Inbox message id
```

Token, OTP, password, connection string ve diğer secret'lar loglanmaz.